adapter.py — technical

src/moe_pruning/models/olmoe/adapter.py · 324 lines · moe_pruning.models.olmoe.adapter

Purpose

Adapter for HF `OlmoeForCausalLM` in transformers 5, where each layer's experts are one `OlmoeExperts` module holding `gate_up_proj [E, 2I, D]` and `down_proj [E, D, I]`.

Data and structure

- Decoder layer: `h = x + attn(norm(x)); out = h + mlp(post_attention_layernorm(h))`.
- `mlp.gate` (router) returns `(logits [T,E], weights [T,K], indices [T,K])`; `mlp.experts(x, indices, weights)`.

Formulas

```
p = softmax(W x) in float32; (w, idx) = top-k(p); if norm_topk_prob: w = w / sum(w)
y_t = sum_k w[t,k] * E_idx[t,k](x_t), E_e(x) = down_e( silu(gate_e x) * up_e x )
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class OlmoeAdapter`

lines 45-324 · bases MoEAdapter

MoEAdapter for OLMoE with transformers 5 fused experts.

`OlmoeAdapter.discover_layers`

```
def discover_layers(self, model: nn.Module) -> list[MoELayerInfo]
```

lines 49-80 · raises RuntimeError

Find all sparse MoE layers by inspecting the loaded model.

1. A MoE block has `gate` (with `top_k` and `weight`) and `experts` (with `gate_up_proj`, `down_proj`); also matches an already pruned `PrunedFusedExperts` container.
2. Layer id from `model.layers.{i}.mlp` -> `layers.{i}`; stack `decoder`; dimensions from the router and `down_proj`; capacity None.

`OlmoeAdapter.get_moe_module`

```
def get_moe_module(self, model, layer)
```

lines 85-86

The sparse MoE module of a layer.

`OlmoeAdapter.get_router`

```
def get_router(self, model, layer)
```

lines 88-89

The router module of a layer.

`OlmoeAdapter.get_expert`

```
def get_expert(self, model, layer, expert_id)
```

lines 94-100 · raises IndexError

Module computing one expert (original, view or replacement).

1. Pruned container -> `container.expert(e)` (row view or replacement). Original container -> `FusedExpertView(experts, e)`.

`OlmoeAdapter.build_original_expert`

```
def build_original_expert(self, model, layer)
```

lines 102-105

New (uninitialised) expert module of the model's original expert class.

`OlmoeAdapter.expert_state_prefix`

```
def expert_state_prefix(self, layer, expert_id)
```

lines 107-108

State-dict key prefix of an expert slot (ends with a dot).

`OlmoeAdapter.replace_experts_in_layer`

```
def replace_experts_in_layer(self, model, layer, modules)
```

lines 110-123 · raises ValueError

Install `modules` in their expert slots; return the number of removed parameters.

1. Reject experts already replaced and refuse to remove every expert.
2. Removed parameters = number of new removals * (rows of `gate_up_proj[0]` + `down_proj[0]`).
3. `PrunedFusedExperts.compact(experts, removed)` copies only retained rows; insert each replacement into `compact.replaced[str(e)]`; assign `mlp.experts = compact` (original tensors become garbage).

`OlmoeAdapter.set_expert`

```
def set_expert(self, model, layer, expert_id, module)
```

lines 125-134

Install `module` in the expert slot and return the previous module.

1. Copy the expert's rows into a standalone `OlmoeReferenceExpert` (returned as 'previous module'), then delegate to `replace_experts_in_layer` for that single expert.

`OlmoeAdapter.removed_weight_violations`

```
def removed_weight_violations(self, state_shapes, layer, removed)
```

lines 136-144

Names of stored tensors that still hold weights of removed experts.

1. Default prefix check plus: stored `gate_up_proj` must have exactly `E - len(removed)` rows.

`OlmoeAdapter.validate_loaded`

```
def validate_loaded(self, model, layers)
```

lines 146-155 · raises RuntimeError

Called by the checkpoint loader after weights are loaded; raise on inconsistency.

1. For each pruned container compare the loaded `retained_expert_ids` buffer with the row order implied by the manifest.

`OlmoeAdapter.load_original_expert`

```
def load_original_expert(self, reader, model, layer, expert_id)
```

lines 157-173 · raises KeyError

Read one expert from the hub layout (per expert) or the fused layout.

1. Hub layout: read `{module_path}.experts.{e}.gate_proj.weight`, `up_proj.weight`, `down_proj.weight`; `gate_up = cat([gate, up], 0)` (transformers' merge order).
2. Fused layout: slice row e of `gate_up_proj` / `down_proj`. Load into `OlmoeReferenceExpert`.

`OlmoeAdapter.route`

```
def route(self, model, layer, moe_input, apply_mask=True)
```

lines 178-187

Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.

1. Call the router on `x.reshape(-1, D)` with the mask flag set as requested; reshape indices/weights to `[..., K]`.

`OlmoeAdapter._masked_routing_hook`

```
def _masked_routing_hook(router, _args, output)
```

lines 190-207

Top-k over the non-masked experts; gates are the original probabilities.

1. Recompute float32 softmax from the logits; masked experts get -1 so top-k picks among the rest; weights = original probabilities of the chosen experts, renormalised only if `norm_topk_prob`; cast to logits dtype.

`OlmoeAdapter.set_routing_mask`

```
def set_routing_mask(self, model, layer, expert_ids)
```

lines 209-219 · raises ValueError

Make `expert_ids` unreachable: their tokens go to the best remaining expert.

1. At least `top_k` experts must remain reachable; store the masked ids; register/remove the prepend hook as needed.

`OlmoeAdapter.get_routing_mask`

```
def get_routing_mask(self, model, layer)
```

lines 221-222

Experts currently made unreachable by the reroute mask.

`OlmoeAdapter.run_moe_layer`

```
def run_moe_layer(self, model, layer, moe_input)
```

lines 224-225

Execute only the sparse MoE module on `moe_input` ([B,S,D]).

`OlmoeAdapter.instrument`

```
def instrument(self, model, layers, callback: RecordCallback) -> Iterator[None]
```

lines 231-239

Call `callback` with a `MoEForwardRecord` per layer forward.

`OlmoeAdapter._register`

```
def _register(self, model, layer, callback)
```

lines 241-289

1. `post_attention_layernorm` pre-hook -> residual `h`; `mlp` pre-hook -> MoE input; router hook -> weights and indices.
2. `mlp` forward hook: for each distinct expert id, select (token, slot) pairs, recompute the un-gated expert output with `get_expert(e)` under `no_grad` (the fused kernel does not expose it), build the record, call the callback.

`OlmoeAdapter.describe`

```
def describe(self, model)
```

lines 294-324

Machine-readable architecture report.

Errors raised

Function	Exception	Message (f-string fields in braces)
`discover_layers`	RuntimeError	Unexpected MoE block location {path}
`get_expert`	IndexError	Expert {expert_id} out of range for {layer.layer_id}
`replace_experts_in_layer`	ValueError	{layer.layer_id}: experts {sorted(overlap)} are already replaced

Function	Exception	Message (f-string fields in braces)
`replace_experts_in_laye r`	ValueError	Refusing to replace every expert of a layer
`validate_loaded`	RuntimeError	{layer.layer_id}: checkpoint rows belong to experts {stored}, manifest implies {experts.retained_ids()}
`load_original_expert`	KeyError	Original weights of {layer.layer_id} expert {expert_id} not found
`set_routing_mask`	ValueError	At least top_k={layer.top_k} experts must remain reachable